

```
In [ ]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader
import random
import matplotlib.pyplot as plt
import marimo as mo
```

```
In [ ]: # define the cnn architecture
class FenceCNN(nn.Module):
    def __init__(self):
        super(FenceCNN,self).__init__()
        # feature extraction (it is same for all conv layers)
        self.conv1=nn.Conv2d(1,8,kernel_size=3,padding=1)
        self.conv2=nn.Conv2d(8,16,kernel_size=3,padding=1)
        self.pool=nn.MaxPool2d(2,2)

        # classification layers mapping features to a single probability
        self.fc1=nn.Linear(16*16*16,64)
        self.fc2=nn.Linear(64,1)

    def forward(self,x):
        # apply conv1, relu activation, and max pooling
        x=self.pool(torch.relu(self.conv1(x)))
        # apply conv2, relu activation, and max pooling
        x=self.pool(torch.relu(self.conv2(x)))
        # flatten the 2d feature maps into a 1d vector
        x=x.view(x.size(0),-1)
        # apply fully connected layers with sigmoid for binary output (0 to 1)
        x=torch.relu(self.fc1(x))
        x=torch.sigmoid(self.fc2(x))
        return x
```

```
In [ ]: # setup image transformations to resize and normalize
t_folders=transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((64,64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5],std=[0.5])
])

# load datasets from directory structure
train_dset=ImageFolder(root="a7/dset/training/",transform=t_folders)
val_dset=ImageFolder(root="a7/dset/validation/",transform=t_folders)
test_dset=ImageFolder(root="a7/dset/testing/",transform=t_folders)

# create iterable data loaders for batching
train_loader=DataLoader(train_dset,batch_size=140,shuffle=True)
val_loader=DataLoader(val_dset,batch_size=30,shuffle=False)
test_loader=DataLoader(test_dset,batch_size=30,shuffle=False)

class_names=train_dset.classes
```

```

# initialize hardware device, model, loss function, and optimizer
device="cuda" if torch.cuda.is_available() else "cpu"
model=FenceCNN().to(device)
criterion=nn.BCELoss()
optimizer=optim.Adam(model.parameters(),lr=0.0001)
epochs=30

```

```
In [ ]: print("Starting training...")
```

```

# main training loop
for epoch in range(epochs):
    model.train()
    correct=0
    total=0

    # iterate through training batches
    for images,labels in train_loader:
        images=images.to(device)
        labels=labels.float().unsqueeze(1).to(device)

        # forward pass
        outputs=model(images)
        loss=criterion(outputs,labels)

        # backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # calculate accuracy metrics
        preds=(outputs>0.5).float()
        correct+=(preds==labels).sum().item()
        total+=labels.size(0)

    train_acc=100*correct/total

    # validation phase
    model.eval()
    val_correct=0
    val_total=0

    # disable gradient calculation for evaluation
    with torch.no_grad():
        for images,labels in val_loader:
            images=images.to(device)
            labels=labels.float().unsqueeze(1).to(device)

            outputs=model(images)
            preds=(outputs>0.5).float()

            val_correct+=(preds==labels).sum().item()
            val_total+=labels.size(0)

    val_acc=100*val_correct/val_total
    print(f"Epoch {epoch+1:02d} | Train Acc: {train_acc:.2f}% | Val Acc: {va

```

```

# testing phase to evaluate final generalized performance
model.eval()
test_correct=0
test_total=0

with torch.no_grad():
    for images, labels in test_loader:
        images=images.to(device)
        labels=labels.float().unsqueeze(1).to(device)

        outputs=model(images)
        preds=(outputs>0.5).float()

        test_correct+=(preds==labels).sum().item()
        test_total+=labels.size(0)

test_acc=100*test_correct/test_total
print(f"Final Test Accuracy: {test_acc:.2f}%")

```

Starting training...

```

Epoch 01 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 02 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 03 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 04 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 05 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 06 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 07 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 08 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 09 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 10 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 11 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 12 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 13 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 14 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 15 | Train Acc: 50.00% | Val Acc: 73.33%
Epoch 16 | Train Acc: 91.43% | Val Acc: 50.00%
Epoch 17 | Train Acc: 50.71% | Val Acc: 50.00%
Epoch 18 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 19 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 20 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 21 | Train Acc: 50.00% | Val Acc: 50.00%
Epoch 22 | Train Acc: 50.00% | Val Acc: 60.00%
Epoch 23 | Train Acc: 69.29% | Val Acc: 86.67%
Epoch 24 | Train Acc: 97.86% | Val Acc: 53.33%
Epoch 25 | Train Acc: 75.00% | Val Acc: 50.00%
Epoch 26 | Train Acc: 52.14% | Val Acc: 50.00%
Epoch 27 | Train Acc: 50.71% | Val Acc: 50.00%
Epoch 28 | Train Acc: 52.14% | Val Acc: 53.33%
Epoch 29 | Train Acc: 67.14% | Val Acc: 90.00%
Epoch 30 | Train Acc: 93.57% | Val Acc: 90.00%
Final Test Accuracy: 96.67%

```

```

In [ ]: # random prediction visualizer
idx=random.randint(0,len(test_dset)-1)
image_tensor,true_label_idx=test_dset[idx]

```

```

# run inference on the single selected image
with torch.no_grad():
    input_img=image_tensor.unsqueeze(0).to(device)
    output=model(input_img)
    prob=output.item()

# map numerical output back to class string
pred_idx=1 if prob>0.5 else 0
predicted_class=class_names[pred_idx]
true_class=class_names[true_label_idx]

print(f"Prediction: {predicted_class} (Probability: {prob:.4f})")
print(f"True Label: {true_class}")

```

Prediction: inside (Probability: 0.4992)
True Label: inside

```

In [ ]: ## Model Evaluation Details ##

from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

def evaluate_validation_detailed(model, dataloader, class_names, device):
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            preds = (outputs > 0.5).float().cpu().numpy()
            labels = labels.cpu().numpy()
            all_preds.extend(preds.flatten())
            all_labels.extend(labels.flatten())
    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)
    accuracy = (all_preds == all_labels).mean() * 100

## outputs ##
print("\n-----Validation Performance-----")
print(f"Accuracy: {accuracy:.2f}%\n")

cm = confusion_matrix(all_labels, all_preds)

print("Confusion Matrix:")
print(cm)
print()

print("Classification Report:")
print(classification_report(all_labels, all_preds, target_names=class_na

return accuracy, cm

```

```
## model performance output details
accuracy, confusion_matrix = evaluate_validation_detailed(
    model,
    val_loader,
    class_names,
    device
)
```

-----Validation Performance-----

Accuracy: 90.00%

Confusion Matrix:

```
[[15  0]
 [ 3 12]]
```

Classification Report:

	precision	recall	f1-score	support
inside	0.83	1.00	0.91	15
outside	1.00	0.80	0.89	15
accuracy			0.90	30
macro avg	0.92	0.90	0.90	30
weighted avg	0.92	0.90	0.90	30

Model Architecture - CNN for dot position w.r.t fence

Input Transformation & Convolutional Feature Extraction: Input Image (64x64x1) -> Convolution 1 (Outputs 8 channels) -> MaxPool2d (Reduces dimensions to 32x32) -> Convolution 2 (Outputs 16 channels) -> MaxPool2d (Reduces dimensions to 16x16). (Note: Kernel 3x3, ReLU activation, and MaxPool2d 2x2 are the same for all convolutional blocks).

Flattening Step (Connecting Convolutions to Dense Layers): Output of Conv 2 -> 3D Tensor of dimensions [16 channels, 16 height, 16 width] -> Flatten Layer -> Unrolls into a 1D array of 4096 total nodes ($16 * 16 * 16 = 4096$).

Fully Connected (Hidden) Layers (Standard Extrapolation for Binary Classification): Input from Flatten -> Fully Connected Hidden Layer 1 (Linear Layer) -> Receives 4096 nodes -> Outputs a compressed feature representation (typically 128 or 256 nodes) -> ReLU activation -> Fully Connected Hidden Layer 2 (Optional but common) -> Receives 128 nodes -> Outputs 64 nodes -> ReLU activation.

Final Output Nodes: Hidden Layer Output -> Final Linear Layer -> Outputs 1 node (using Sigmoid activation to yield a probability between 0 and 1) OR Outputs 2 nodes (using Softmax activation to yield discrete probabilities for "inside" and "outside").